

1. Swap two variables without using a third variable.
2. Check if a number is prime.
3. Check if a string is a palindrome.
4. Reverse a string / a list without using built-in reverse().
5. Find the factorial of a number (using loop and using recursion).
6. Find the Fibonacci series up to n terms.
7. Count vowels/consonants in a string.
8. Find the largest/smallest number in a list without using max()/min().
9. Remove duplicates from a list.
10. Check if two strings are anagrams.
11. Find the frequency of each character/word in a string.
12. Sort a list without using sort() (implement bubble sort/selection sort).
13. Find the second largest number in a list.
14. Check if a number is an Armstrong number.
15. Merge two dictionaries.
16. Convert a list of tuples into a dictionary.
17. Find common elements between two lists.
18. Flatten a nested list.
19. Write a program to count the occurrences of each element in a list using a dictionary.
20. Difference between is and ==, mutable vs immutable types, *args vs **kwargs, list vs tuple vs set vs dict.

OOP Basics-----

1. Create a class Student with attributes name and marks; add a method to display details.
2. Explain and demonstrate the 4 pillars of OOP: Encapsulation, Abstraction, Inheritance, Polymorphism — with a simple code example for each.
3. Difference between class variables and instance variables (with example).
4. Difference between __init__ and __new__.
5. What is self? Why is it needed?
6. Implement single inheritance — e.g., Animal → Dog class, override a method.
7. Implement multiple inheritance and explain Method Resolution Order (MRO).
8. What is method overriding vs method overloading? (Python doesn't support true overloading — explain how it's mimicked with default args.)
9. Difference between classmethod, staticmethod, and instance method — write one example of each.
10. What are dunder/magic methods? Implement __str__ and __repr__ for a class.
11. What is encapsulation? Demonstrate private (__var) and protected (_var) attributes.
12. What is an abstract class? Implement one using the abc module.
13. Explain polymorphism with a real example (e.g., different classes having a speak() method).
14. What is the difference between a class and an object?
15. Implement operator overloading — e.g., overload + for a Vector class using __add__.
16. What is composition vs inheritance ("has-a" vs "is-a" relationship)?
17. Create a BankAccount class with deposit/withdraw methods and balance validation — a very common OOP coding question.
18. What happens if you don't define __init__ in a class?
19. Explain super() and when/why it's used.
20. Can you instantiate an abstract class in Python? Why or why not?